

Índice general

Manual de Operaciones: Pipeline CI/CD con Jenkins	2
Portada	2
1. Requisitos Previos	2
1.1 Requisitos de Hardware	2
1.2 Requisitos de Software	3
1.3 Cuentas Requeridas	3
2. Instalación de Jenkins	3
2.1 Instalación Mediante Docker (Recomendado)	3
2.2 Instalación Manual (WAR)	4
2.3 Asistente de Configuración Inicial	4
3. Configuración del Repositorio GitHub	5
3.1 Crear Repositorio en GitHub	5
3.2 Clonar Repositorio Localmente	5
3.3 Agregar Archivos del Proyecto	5
3.4 Realizar Primer Commit y Push	6
4. Configuración del Jenkinsfile	6
4.1 Estructura del Pipeline	6
4.2 Explicación de Etapas	8
4.3 Configuración de Post-Acciones	8
5. Creación del Pipeline en Jenkins	9
5.1 Crear Nuevo Trabajo Pipeline	9
5.2 Configurar Repositorio Git	9
5.3 Configurar Gatillo de Build	9
5.4 Especificar Ruta del Jenkinsfile	9
5.5 Guardar Configuración	10
6. Configuración de Webhooks en GitHub	10
6.1 Acceder a Configuración de Webhooks	10
6.2 Crear Nuevo Webhook	10
6.3 Usar ngrok para Webhooks Locales	10
6.4 Verificar Entrega del Webhook	11
7. Configuración de Notificaciones por Email	11
7.1 Configurar SMTP en Jenkins	11
7.2 Obtener Contraseña de Aplicación de Gmail	11
7.3 Configurar Notificaciones en el Jenkinsfile	12
7.4 Probar Configuración de Email	12
8. Ejecución del Pipeline	12
8.1 Disparar Pipeline Manualmente	12
8.2 Disparar Pipeline Automáticamente	12
8.3 Monitoreo de Ejecución - Etapa por Etapa	13
8.4 Vista General del Pipeline	14
9. Verificación del Despliegue	14
9.1 Acceder a la Aplicación	14
9.2 Verificar Endpoint de Salud	14
9.3 Pruebas de Endpoints	15
9.4 Verificar Contenedor Docker	15

10. Resultados de Pruebas	16
10.1 Acceder a Reportes en Jenkins	16
10.2 Reporte JUnit	16
10.3 Reporte de Cobertura	16
10.4 Resumen de Resultados	16
11. Video Demostrativo	16
11.1 Contenido del Video	16
11.2 Enlace del Video	17
12. Solución de Problemas Comunes	18
12.1 Pipeline No Se Dispara Automáticamente	18
12.2 Error en Etapa de Checkout	18
12.3 Error en Etapa de Setup Environment	19
12.4 Error en Etapa de Build (Docker)	19
12.5 Error en Etapa de Test	20
12.6 Error en Etapa de Deploy	20
12.7 Notificaciones por Email No Llegan	20
13. Conclusiones	21
Apéndices	22
A. Referencia de Comandos Útiles	22
B. Estructura de Directorios Recomendada	22
C. Checklist de Verificación	23

Manual de Operaciones: Pipeline CI/CD con Jenkins

Portada

Pipeline CI/CD con Jenkins

Asignatura: Gestión de la Configuración del Software

Profesor: Ph.D. Franklin Parrales-Bravo

Universidad: Universidad de Guayaquil

Integrantes del Grupo D: - Aguilar Villafuerte Daniel Mateo - Arroba Carrillo Omar Andres - Ayovi Villafuerte Camillie Thais - Cordova Viteri Erick Alejandro - Tipán Antón Cesar Alexander

Fecha: Mayo 2026

1. Requisitos Previos

1.1 Requisitos de Hardware

- **Procesador:** Intel Core i5 o superior (o equivalente AMD)
- **Memoria RAM:** Mínimo 8 GB (recomendado 16 GB)

- **Espacio en disco:** Mínimo 20 GB de espacio libre
- **Sistema Operativo:** Windows 10/11 (con WSL2), macOS, o Linux

1.2 Requisitos de Software

- **Java Development Kit (JDK) 17:** Necesario para ejecutar Jenkins
 - Descargar desde: <https://www.oracle.com/java/technologies/downloads/#java17>
 - Verificar instalación: `java -version`
- **Jenkins:** Servidor de automatización CI/CD
 - Versión: 2.440 o superior
 - Modo de instalación: WAR o Docker
- **Docker:** Para contenerizar la aplicación y las herramientas
 - Versión: 24.0 o superior
 - Descargar desde: <https://www.docker.com/products/docker-desktop>
- **Git:** Sistema de control de versiones
 - Versión: 2.40 o superior
 - Descargar desde: <https://git-scm.com/>
- **Python:** Lenguaje de programación para la aplicación
 - Versión: 3.11 o superior
 - Descargar desde: <https://www.python.org/>
- **ngrok:** Herramienta para exponer servidores locales a internet
 - Versión: 3.0 o superior
 - Descargar desde: <https://ngrok.com/>
 - Necesario para webhooks en entornos locales
- **curl o Postman:** Herramientas para pruebas de API (opcional)

1.3 Cuentas Requeridas

- **Cuenta de GitHub:** Para albergar el repositorio del proyecto
 - Registro: <https://github.com/signup>
- **Cuenta de Gmail:** Para notificaciones por correo electrónico desde Jenkins
 - Es recomendable usar una contraseña de aplicación específica
- **Cuenta de YouTube:** Para subir el video demostrativo del proyecto

2. Instalación de Jenkins

2.1 Instalación Mediante Docker (Recomendado)

Jenkins se puede instalar de forma rápida y eficiente usando Docker:

```
docker run -d -p 8080:8080 -p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
--name jenkins \
jenkins/jenkins:ls
```

Explicación de parámetros: - -d: Ejecutar en segundo plano - -p 8080:8080: Puerto web de Jenkins - -p 50000:50000: Puerto para agentes Jenkins - -v jenkins_home:/var/jenkins_home: Volumen persistente para datos - -v /var/run/docker.sock:/var/run/docker.sock: Acceso a Docker desde Jenkins

2.2 Instalación Manual (WAR)

1. Descargar Jenkins desde <https://www.jenkins.io/download/>
2. Instalar Java JDK 17 previamente
3. Ejecutar el archivo WAR:

```
java -jar jenkins.war
```
4. Acceder a <http://localhost:8080>

2.3 Asistente de Configuración Inicial

1. Obtener contraseña inicial:

```
docker logs jenkins | grep -A 5 "Please use the following password"
```

O en instalación manual, revisar la consola de salida.

[Captura de pantalla: Pantalla inicial de Jenkins mostrando el campo para contraseña de desbloqueo]

2. Seleccionar complementos sugeridos:

- Haz clic en “Install suggested plugins”

[Captura de pantalla: Pantalla de selección de complementos sugeridos de Jenkins]

3. Instalar complementos adicionales: Una vez instalados los complementos sugeridos, ve a **Manage Jenkins → Plugin Manager** e instala:

- Git Plugin
- Pipeline
- Docker Pipeline
- Email Extension Plugin
- JUnit Plugin
- Cobertura Plugin
- GitHub Integration Plugin

[Captura de pantalla: Panel de Plugin Manager mostrando complementos instalados]

4. Crear usuario administrador:

- Completar formulario con datos de usuario
- Nombre de usuario recomendado: `admin`
- Establecer contraseña segura

[Captura de pantalla: Formulario de creación de usuario administrador]

5. Configurar URL de Jenkins:

- URL recomendada: <http://localhost:8080/>
- Esta URL es importante para los webhooks

[Captura de pantalla: Pantalla de configuración de URL de Jenkins]

6. Confirmar instalación:

- Haz clic en “Start using Jenkins”
- Deberías ver el dashboard de Jenkins

[Captura de pantalla: Dashboard principal de Jenkins después de completar la instalación]

3. Configuración del Repositorio GitHub

3.1 Crear Repositorio en GitHub

1. Acceder a <https://github.com/new>
2. Completar los siguientes datos:
 - **Repository name:** GCS-Proyecto-P1
 - **Description:** Pipeline CI/CD con Jenkins - Grupo D
 - **Visibility:** Public
 - **Initialize with README:** Dejar sin marcar (lo haremos manualmente)

[Captura de pantalla: Formulario de creación de repositorio en GitHub]

3. Haz clic en “Create repository”

3.2 Clonar Repositorio Localmente

```
git clone https://github.com/TU_USUARIO/GCS-Proyecto-P1.git
cd GCS-Proyecto-P1
```

3.3 Agregar Archivos del Proyecto

Asegúrate de que la estructura del proyecto sea la siguiente:

```
GCS-Proyecto-P1/
  app/
    __init__.py
    main.py
  tests/
    __init__.py
    test_main.py
    conftest.py
  docs/
    INFORME_TECNICO.md
    MANUAL_OPERACIONES.md
  Dockerfile
  docker-compose.yml
  Jenkinsfile
  requirements.txt
  .gitignore
```

README.md

3.4 Realizar Primer Commit y Push

```
git add .
git commit -m "Initial project setup with Jenkinsfile and Docker configuration"
git branch -M main
git push -u origin main
```

[Captura de pantalla: Output del push a GitHub mostrando archivos cargados]

4. Configuración del Jenkinsfile

4.1 Estructura del Pipeline

El Jenkinsfile define las etapas del pipeline CI/CD. A continuación se describe cada etapa:

```
pipeline {
    agent any

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    stages {
        stage('Checkout') {
            steps {
                echo 'Clonando repositorio...'
                checkout scm
            }
        }

        stage('Setup Environment') {
            steps {
                echo 'Configurando ambiente...'
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Lint') {
```

```

    steps {
        echo 'Ejecutando análisis de código...'
        sh '''
            . venv/bin/activate
            pip install flake8
            flake8 app/ tests/ --count --select=E9,F63,F7,F82 --show-source --statistics
        '''
    }
}

stage('Build') {
    steps {
        echo 'Construyendo imagen Docker...'
        sh '''
            docker build -t gcs-proyecto-p1:latest .
            docker tag gcs-proyecto-p1:latest gcs-proyecto-p1:${BUILD_NUMBER}
        '''
    }
}

stage('Test') {
    steps {
        echo 'Ejecutando pruebas unitarias...'
        sh '''
            . venv/bin/activate
            pip install pytest pytest-cov
            pytest tests/ --junitxml=results.xml --cov=app --cov-report=xml
        '''
    }
}

stage('Deploy') {
    steps {
        echo 'Desplegando aplicación...'
        sh '''
            docker stop gcs-app || true
            docker rm gcs-app || true
            docker run -d -p 5000:5000 --name gcs-app gcs-proyecto-p1:latest
            sleep 5
            curl -f http://localhost:5000/health || exit 1
        '''
    }
}

post {
    always {
        junit 'results.xml'
    }
}

```

```

        publishHTML([
            reportDir: 'htmlcov',
            reportFiles: 'index.html',
            reportName: 'Coverage Report'
        ])
    }
    success {
        echo 'Pipeline ejecutado exitosamente'
        emailext(
            subject: "Pipeline exitoso: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
            body: "El pipeline se completó exitosamente. Ver detalles en: ${env.BUILD_URL}",
            to: '${DEFAULT_RECIPIENTS}'
        )
    }
    failure {
        echo 'Pipeline falló'
        emailext(
            subject: "Pipeline fallido: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
            body: "El pipeline falló. Ver detalles en: ${env.BUILD_URL}",
            to: '${DEFAULT_RECIPIENTS}'
        )
    }
}
}
}

```

4.2 Explicación de Etapas

Etapa	Descripción
Checkout	Clona el repositorio de GitHub en el workspace de Jenkins
Setup Environment	Crea ambiente Python virtual e instala dependencias
Lint	Valida la calidad del código usando flake8
Build	Construye la imagen Docker de la aplicación
Test	Ejecuta pruebas unitarias y genera reporte de cobertura
Deploy	Despliega la aplicación en contenedor Docker

4.3 Configuración de Post-Acciones

El bloque `post` incluye: - **always:** Publica reportes JUnit y de cobertura - **success:** Envía notificación por email en caso de éxito - **failure:** Envía notificación por email en caso de fallo

5. Creación del Pipeline en Jenkins

5.1 Crear Nuevo Trabajo Pipeline

1. En el dashboard de Jenkins, haz clic en **New Item**

[Captura de pantalla: Dashboard de Jenkins con opción 'New Item' destacada]

2. Ingresar nombre del trabajo: `GCS-Proyecto-P1-Pipeline`
3. Seleccionar tipo: **Pipeline**

[Captura de pantalla: Formulario de creación de nuevo trabajo con tipo 'Pipeline' seleccionado]

4. Haz clic en **OK**

5.2 Configurar Repositorio Git

1. En la sección **Pipeline**, seleccionar **Pipeline script from SCM**

[Captura de pantalla: Sección de configuración de Pipeline mostrando opciones]

2. Cambiar SCM a **Git**
3. Ingresar la URL del repositorio:

`https://github.com/TU_USUARIO/GCS-Proyecto-P1.git`

[Captura de pantalla: Campo de URL del repositorio Git en Jenkins]

4. Si es repositorio privado, agregar credenciales:
 - Haz clic en **Add → Jenkins**
 - Seleccionar **Username with password**
 - Ingresar tu usuario y token de GitHub (PAT)

[Captura de pantalla: Formulario de agregar credenciales de GitHub]

5. En **Branches to build**, verificar que esté establecido en `*/main`

5.3 Configurar Gatillo de Build

1. Ir a la sección **Build Triggers**
2. Marcar **GitHub hook trigger for GITScm polling**

[Captura de pantalla: Sección Build Triggers con opción de GitHub hook seleccionada]

3. Esta opción permite que GitHub dispare el pipeline automáticamente mediante webhooks

5.4 Especificar Ruta del Jenkinsfile

1. En la sección **Pipeline**, bajo **Definition**, verificar que esté seleccionado **Pipeline script from SCM**
2. En **Script Path**, ingresar: `Jenkinsfile`

[Captura de pantalla: Campo Script Path con 'Jenkinsfile' especificado]

5.5 Guardar Configuración

1. Haz clic en **Save** al final de la página
2. Jenkins validará la configuración

[Captura de pantalla: Página de detalles del pipeline después de guardar]

6. Configuración de Webhooks en GitHub

6.1 Acceder a Configuración de Webhooks

1. En GitHub, ir al repositorio GCS-Proyecto-P1
2. Haz clic en **Settings** (esquina superior derecha)

[Captura de pantalla: Página de repositorio con pestaña 'Settings' visible]

3. En el menú lateral, haz clic en **Webhooks**

[Captura de pantalla: Menú lateral de Settings con opción 'Webhooks' destacada]

6.2 Crear Nuevo Webhook

1. Haz clic en **Add webhook**

[Captura de pantalla: Página de Webhooks con botón 'Add webhook']

2. Configurar los siguientes campos:

Payload URL: - Si Jenkins está en red pública: `http://TU_IP_JENKINS:8080/github-webhook/`
- Si Jenkins está en localhost, usar ngrok (ver sección 6.3)

[Captura de pantalla: Campo Payload URL con ejemplo de URL completado]

Content type: Seleccionar `application/json`

Events: Seleccionar **Just the push event**

[Captura de pantalla: Sección de selección de eventos con 'Just the push event' marcado]

Active: Mantener marcado

3. Haz clic en **Add webhook**

[Captura de pantalla: Botón 'Add webhook' completando la configuración]

6.3 Usar ngrok para Webhooks Locales

Si Jenkins está en localhost, usar ngrok para exponerlo a internet:

1. Descargar ngrok desde <https://ngrok.com/>
2. Autenticarse: `ngrok config add-authtoken TU_TOKEN`
3. Exponer Jenkins:

```
ngrok http 8080
```

4. Ngrok proporcionará una URL como: `https://xxxx-xxx-xxx-xxxx.ngrok.io`

5. Usar como Payload URL en GitHub:

```
https://xxxx-xxx-xxx-xxxx.ngrok.io/github-webhook/
```

[Captura de pantalla: Output de ngrok mostrando URL pública generada]

6.4 Verificar Entrega del Webhook

1. En la página de Webhooks de GitHub, hacer scroll hacia abajo
2. En **Recent Deliveries**, debería haber una entrada con estado 200

[Captura de pantalla: Sección 'Recent Deliveries' mostrando webhook entregado exitosamente]

3. Si está en rojo, revisar los logs de Jenkins en **Manage Jenkins → System Log**

7. Configuración de Notificaciones por Email

7.1 Configurar SMTP en Jenkins

1. En Jenkins, ir a **Manage Jenkins → Configure System**

[Captura de pantalla: Panel 'Manage Jenkins' con opción 'Configure System' visible]

2. Buscar sección **Email Notification** o desplazarse hacia el final
3. Configurar los siguientes parámetros:

SMTP server: `smtp.gmail.com`

SMTP port: 587

Default user email suffix: `@gmail.com`

[Captura de pantalla: Sección de configuración SMTP con campos completados]

4. Marcar **Use SMTP Authentication**
5. Ingresar credenciales de Gmail:
 - **User:** Tu correo Gmail completo
 - **Password:** Contraseña de aplicación de Gmail (no la contraseña de cuenta)

[Captura de pantalla: Campo de credenciales SMTP con usuario y contraseña]

6. Marcar **Use TLS**

7.2 Obtener Contraseña de Aplicación de Gmail

1. Ir a `https://myaccount.google.com/`
2. Seleccionar **Security** en el menú lateral

3. Buscar **App passwords** (requiere autenticación 2FA)
4. Seleccionar **Mail y Windows Computer**
5. Google generará una contraseña de 16 caracteres
6. Copiar esta contraseña en Jenkins

[Captura de pantalla: Panel de seguridad de Google con opción 'App passwords']

7.3 Configurar Notificaciones en el Jenkinsfile

El Jenkinsfile ya incluye la configuración de email en el bloque post:

```
emailext(  
    subject: "Pipeline exitoso: ${env.JOB_NAME} #${env.BUILD_NUMBER}",  
    body: "El pipeline se completó exitosamente. Ver detalles en: ${env.BUILD_URL}",  
    to: '${DEFAULT_RECIPIENTS}'  
)
```

Los correos se enviarán automáticamente después de cada build.

7.4 Probar Configuración de Email

1. Hacer un commit y push a GitHub
2. Esperar a que se ejecute el pipeline
3. Revisar la bandeja de entrada de email configurada

[Captura de pantalla: Correo de notificación de Jenkins en bandeja de entrada]

8. Ejecución del Pipeline

8.1 Disparar Pipeline Manualmente

Primera ejecución (si no hay webhook aún):

1. En Jenkins, ir al trabajo **GCS-Proyecto-P1-Pipeline**
2. Haz clic en **Build Now**

[Captura de pantalla: Página del pipeline con botón 'Build Now' destacado]

3. El pipeline iniciará inmediatamente

8.2 Disparar Pipeline Automáticamente

Una vez configurados los webhooks:

1. Hacer un commit y push a GitHub:

```
git add .  
git commit -m "Test webhook trigger"  
git push origin main
```

2. GitHub enviará automáticamente una notificación a Jenkins

3. El pipeline se disparará automáticamente en segundos

[Captura de pantalla: Webhook enviado desde GitHub a Jenkins]

8.3 Monitoreo de Ejecución - Etapa por Etapa

Etapa: Checkout

```
[INFO] Clonando repositorio...
[INFO] Cloning git repository...
[INFO] Cloning into '/var/jenkins_home/workspace/GCS-Proyecto-P1-Pipeline'...
[INFO] Finished: SUCCESS
```

[Captura de pantalla: Console output mostrando etapa Checkout completada en verde]

Etapa: Setup Environment

```
[INFO] Configurando ambiente...
[INFO] Creating Python virtual environment...
[INFO] Collecting pip...
[INFO] Installing collected packages...
[INFO] Successfully installed all packages
```

[Captura de pantalla: Console output mostrando instalación de dependencias completada]

Etapa: Lint

```
[INFO] Ejecutando análisis de código...
[INFO] Running flake8...
[INFO] ./app/main.py:1:1: F401 'os' imported but unused
[INFO] Total violations: 0 after fixes
```

[Captura de pantalla: Console output mostrando resultados de linting]

Etapa: Build

```
[INFO] Construyendo imagen Docker...
[INFO] Sending build context to Docker daemon...
[INFO] Successfully built gcs-proyecto-p1:latest
[INFO] Successfully tagged gcs-proyecto-p1:6
```

[Captura de pantalla: Console output mostrando construcción de imagen Docker exitosa]

Etapa: Test

```
[INFO] Ejecutando pruebas unitarias...
[INFO] ===== test session starts =====
[INFO] collected 5 items
[INFO] tests/test_main.py::test_health PASSED [ 20%]
```

```
[INFO] tests/test_main.py::test_endpoint_1 PASSED [ 40%]
[INFO] tests/test_main.py::test_endpoint_2 PASSED [ 60%]
[INFO] tests/test_main.py::test_endpoint_3 PASSED [ 80%]
[INFO] tests/test_main.py::test_endpoint_4 PASSED [100%]
[INFO] ===== 5 passed in 2.34s =====
[INFO] Coverage: 92%
```

[Captura de pantalla: Console output mostrando resultados de pruebas con cobertura]

Etapa: Deploy

```
[INFO] Desplegando aplicación...
[INFO] Stopping previous container...
[INFO] Starting new application container...
[INFO] Container started successfully
[INFO] Verifying application health...
[INFO] Health check passed
[INFO] Application deployed and running on port 5000
```

[Captura de pantalla: Console output mostrando despliegue exitoso]

8.4 Vista General del Pipeline

1. En la página del trabajo, hacer clic en el número del build (ej. #1)
2. Seleccionar **Stage View** para ver todas las etapas

[Captura de pantalla: Stage View mostrando todas las etapas en verde (exitosas)]

3. Cada etapa en verde indica ejecución exitosa
4. Haz clic en una etapa para ver detalles específicos

[Captura de pantalla: Detalles de una etapa específica con tiempos de ejecución]

9. Verificación del Despliegue

9.1 Acceder a la Aplicación

Una vez que el pipeline completa la etapa de Deploy, la aplicación está en ejecución:

URL de acceso: `http://localhost:5000`

[Captura de pantalla: Navegador mostrando página principal de la aplicación en localhost:5000]

9.2 Verificar Endpoint de Salud

Ejecutar el siguiente comando para verificar que la aplicación está respondiendo:

```
curl -X GET http://localhost:5000/health
```

Respuesta esperada:

```
{
  "status": "healthy",
  "timestamp": "2026-05-13T15:30:00Z"
}
```

[Captura de pantalla: Terminal mostrando respuesta del endpoint /health en JSON]

9.3 Pruebas de Endpoints

Probar los diferentes endpoints de la aplicación:

Endpoint 1: Información General

```
curl -X GET http://localhost:5000/api/info
```

[Captura de pantalla: Respuesta de endpoint /api/info]

Endpoint 2: Datos de Ejemplo

```
curl -X GET http://localhost:5000/api/data
```

[Captura de pantalla: Respuesta de endpoint /api/data con datos de ejemplo]

Endpoint 3: POST de Datos

```
curl -X POST http://localhost:5000/api/process \
  -H "Content-Type: application/json" \
  -d '{"value": "test"}'
```

[Captura de pantalla: Respuesta de endpoint POST /api/process]

9.4 Verificar Contenedor Docker

Verificar que el contenedor está ejecutándose correctamente:

```
docker ps | grep gcs-app
```

Salida esperada:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
abc123def456	gcs-proyecto-p1:latest	"python app/main.py"	2 minutes ago	Up 2 minutes

[Captura de pantalla: Output de docker ps mostrando contenedor ejecutándose]

Ver logs del contenedor:

```
docker logs gcs-app
```

[Captura de pantalla: Logs del contenedor mostrando aplicación iniciada correctamente]

10. Resultados de Pruebas

10.1 Acceder a Reportes en Jenkins

1. En Jenkins, ir al trabajo **GCS-Proyecto-P1-Pipeline**
2. Hacer clic en el número de build (ej. #1)

[Captura de pantalla: Página de detalles del build con múltiples reportes disponibles]

10.2 Reporte JUnit

1. En la página del build, buscar **Test Result Summary**
2. Se mostrará el número de pruebas pasadas, falló y omitidas

[Captura de pantalla: Test Result Summary mostrando 5 pruebas pasadas sin fallos]

3. Haz clic en **History** para ver tendencias de pruebas

[Captura de pantalla: Gráfico de tendencia de pruebas a lo largo de múltiples builds]

10.3 Reporte de Cobertura

1. En la página del build, buscar **Coverage Report**
2. Haz clic en **Coverage Report** para abrir el reporte detallado

[Captura de pantalla: Coverage Report mostrando 92% de cobertura global]

El reporte incluye: - Cobertura por archivo - Líneas cubiertas vs no cubiertas - Ramas cubiertas

[Captura de pantalla: Detalles de cobertura por archivo con líneas resaltadas]

10.4 Resumen de Resultados

Ejecución #1: - Estado: SUCCESS - Duración: 5 minutos 32 segundos - Pruebas: 5 pasadas, 0 fallos - Cobertura: 92% - Imagen Docker: gcs-proyecto-p1:1

[Captura de pantalla: Panel de resumen del build mostrando todas las métricas]

11. Video Demostrativo

11.1 Contenido del Video

Se ha preparado un video demostrativo que muestra:

1. **Instalación de Jenkins** (0:00 - 2:30)
 - Descarga e instalación de Jenkins
 - Configuración inicial del servidor
 - Instalación de complementos
2. **Configuración del Repositorio GitHub** (2:30 - 5:00)
 - Creación del repositorio

- Estructura del proyecto
 - Push inicial de código
3. **Creación del Pipeline en Jenkins** (5:00 - 8:30)
 - Crear nuevo trabajo Pipeline
 - Configurar conexión a GitHub
 - Configurar gatillos de webhook
 4. **Ejecución del Pipeline** (8:30 - 15:00)
 - Primer build manual
 - Ejecución de cada etapa en detalle
 - Visualización de Stage View
 5. **Verificación de Resultados** (15:00 - 18:00)
 - Pruebas de endpoints de la aplicación
 - Revisión de reportes JUnit y cobertura
 - Verificación de despliegue en Docker
 6. **Notificaciones por Email** (18:00 - 20:00)
 - Configuración de SMTP en Jenkins
 - Prueba de notificación de build exitoso

Duración total: Aproximadamente 20 minutos

11.2 Enlace del Video

[Enlace al video de YouTube: PENDIENTE]

Descripción del video:

Pipeline CI/CD con Jenkins - Demostración Completa

Este video muestra la implementación y ejecución de un pipeline CI/CD completo usando Jenkins, Docker, GitHub y Python. Se demuestra:

- Instalación y configuración de Jenkins
- Integración con GitHub mediante webhooks
- Definición del pipeline en Jenkinsfile
- Automatización de build, test y deploy
- Análisis de resultados y reportes

Curso: Gestión de la Configuración del Software

Profesor: Ph.D. Franklin Parrales-Bravo

Universidad: Universidad de Guayaquil

Grupo: D

Fecha: Mayo 2026

12. Solución de Problemas Comunes

12.1 Pipeline No Se Dispara Automáticamente

Síntoma: El webhook está configurado pero el pipeline no se ejecuta.

Soluciones:

1. **Verificar que el servidor Jenkins es accesible desde GitHub:**

- Si Jenkins está en localhost, verificar que GitHub puede alcanzar la URL
- Usar ngrok si Jenkins está en red local
- Revisar firewall y reglas de seguridad

2. **Verificar configuración del webhook en GitHub:**

En la página del webhook, revisar "Recent Deliveries"
Buscar respuestas con código 200 (exitoso)
Si hay errores, haz clic en la entrega para ver detalles

3. **Revisar logs de Jenkins:**

- Ir a **Manage Jenkins → System Log**
- Buscar mensajes de error relacionados con GitHub
- Aumentar nivel de logging si es necesario

[Captura de pantalla: System Log de Jenkins mostrando errores de webhook]

4. **Validar configuración del trigger en el pipeline:**

- Ir a trabajo **GCS-Proyecto-P1-Pipeline → Configure**
- Verificar que **GitHub hook trigger for GITScm polling** está marcado
- Hacer click en **Save** para revalidar

12.2 Error en Etapa de Checkout

Error típico:

```
ERROR: Error cloning remote repo 'origin'  
hudson.plugins.git.GitException: Command "git clone..." returned status code 128
```

Soluciones:

1. **Verificar URL del repositorio:**

- Debe ser accesible públicamente o tener credenciales configuradas
- Probar URL manualmente: `git clone https://github.com/usuario/repo.git`

2. **Configurar credenciales en Jenkins:**

- Ir a **Credentials → System → Global credentials → Add Credentials**
- Usar token de GitHub (PAT) en lugar de contraseña
- Seleccionar credenciales en configuración del pipeline

[Captura de pantalla: Panel de credenciales de Jenkins con GitHub PAT]

3. **Verificar permisos:**

- Usuario de GitHub debe tener acceso al repositorio
- Token de GitHub debe estar activo

12.3 Error en Etapa de Setup Environment

Error típico:

ERROR: python3: command not found

Soluciones:

1. Instalar Python en agente de Jenkins:

```
# En Linux/WSL
sudo apt-get update
sudo apt-get install python3 python3-venv python3-pip

# En macOS
brew install python3
```

2. Configurar Python como herramienta en Jenkins:

- Ir a **Manage Jenkins → Global Tool Configuration**
- Buscar sección **Python**
- Especificar ruta a Python: `/usr/bin/python3`

[Captura de pantalla: Global Tool Configuration con ruta de Python]

3. Usar imagen Docker con Python preinstalado:

- Cambiar agent a `agent { docker { image 'python:3.11' } }`

12.4 Error en Etapa de Build (Docker)

Error típico:

ERROR: Cannot connect to Docker daemon. Is the docker daemon running?

Soluciones:

1. Verificar que Docker está ejecutándose:

```
docker ps
```

Si no funciona, iniciar Docker.

2. Dar permisos a Jenkins para acceder a Docker:

```
# En Linux
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

3. Usar Docker-in-Docker en Jenkins:

- Configurar Jenkins con soporte DinD
- Usar imagen `jenkins/jenkins:dind`

[Captura de pantalla: Instalación de Docker dentro de contenedor Jenkins]

12.5 Error en Etapa de Test

Error típico:

```
ERROR: ERROR collecting test session
ImportError: No module named 'pytest'
```

Soluciones:

1. Verificar que pytest está en requirements.txt:

```
pytest==7.4.0
pytest-cov==4.1.0
```

2. Instalar dependencias de prueba:

```
. venv/bin/activate
pip install -r requirements.txt
pip install pytest pytest-cov
```

3. Revisar ruta de pruebas en Jenkinsfile:

- Verificar que `pytest tests/` es correcto
- Asegurar que archivo `tests/__init__.py` existe

[Captura de pantalla: Output de pytest mostrando pruebas encontradas y ejecutadas]

12.6 Error en Etapa de Deploy

Error típico:

```
ERROR: Health check failed
curl: (7) Failed to connect to 127.0.0.1 port 5000
```

Soluciones:

1. Verificar que aplicación inicia correctamente:

```
docker run -p 5000:5000 gcs-proyecto-p1:latest
# En otra terminal
curl -f http://localhost:5000/health
```

2. Aumentar tiempo de espera en Jenkinsfile:

```
sleep 10 # Aumentar de 5 a 10 segundos
```

3. Revisar logs del contenedor:

```
docker logs gcs-app
```

[Captura de pantalla: Logs de aplicación mostrando error de inicialización]

12.7 Notificaciones por Email No Llegan

Síntoma: Pipeline se ejecuta pero no se reciben emails.

Soluciones:

1. **Verificar configuración SMTP:**
 - Ir a **Manage Jenkins → Configure System**
 - Revisar que servidor sea `smtp.gmail.com` y puerto sea 587
 - Probar credenciales manualmente con telnet o herramienta similar
2. **Usar contraseña de aplicación de Gmail:**
 - No usar contraseña regular de Google
 - Generar nueva desde <https://myaccount.google.com/apppasswords>
 - Requiere autenticación 2FA habilitada
3. **Revisar spam o filtros:**
 - Buscar email en carpeta de spam
 - Añadir dirección de Jenkins a contactos
4. **Habilitar autenticación 2FA en Gmail:**
 - Necesario para generar contraseñas de aplicación
 - Ir a <https://myaccount.google.com/security>

[Captura de pantalla: Panel de seguridad de Google con 2FA habilitado]

5. **Revisar logs de email en Jenkins:**
 - Ir a **Manage Jenkins → System Log**
 - Filtrar por “mail” o “email”
 - Buscar mensajes de error
-

13. Conclusiones

Este manual proporciona una guía completa para:

- Instalar y configurar Jenkins
- Crear un pipeline CI/CD automatizado
- Integrar con GitHub mediante webhooks
- Ejecutar pruebas y análisis de código
- Desplegar aplicaciones con Docker
- Notificar resultados por email

El pipeline implementado demuestra las mejores prácticas de Gestión de la Configuración del Software:

1. **Integración Continua:** El código se integra automáticamente con cada push
 2. **Automatización:** Todas las etapas se ejecutan sin intervención manual
 3. **Control de Calidad:** Las pruebas se ejecutan automáticamente
 4. **Trazabilidad:** Todos los builds y resultados quedan registrados
 5. **Notificación:** El equipo recibe feedback inmediato de cada cambio
-

Apéndices

A. Referencia de Comandos Útiles

Jenkins

```
curl -I http://localhost:8080      # Verificar Jenkins está activo
curl http://localhost:8080/queue   # Ver cola de builds pendientes
```

Git

```
git log --oneline                  # Ver historial de commits
git branch -a                     # Ver todas las ramas
git diff main origin/main         # Ver diferencias con remoto
```

Docker

```
docker ps -a                      # Listar todos los contenedores
docker logs -f gcs-app            # Ver logs en tiempo real
docker exec -it gcs-app bash      # Acceder a contenedor en ejecución
docker image ls                   # Listar imágenes
docker rmi gcs-proyecto-p1:latest # Eliminar imagen
```

Python

```
python3 -m venv venv              # Crear ambiente virtual
source venv/bin/activate          # Activar ambiente
pip freeze > requirements.txt     # Generar requirements
```

curl / API Testing

```
curl -X GET http://localhost:5000/health
curl -X POST http://localhost:5000/api/endpoint -H "Content-Type: application/json" -d '{...}'
```

B. Estructura de Directorios Recomendada

```
GCS-Proyecto-P1/
├── .github/
│   └── workflows/          # (Opcional) GitHub Actions
├── .gitignore
├── app/
│   ├── __init__.py
│   ├── main.py             # Aplicación principal Flask
│   └── models.py           # (Opcional) Modelos de datos
├── docs/
│   ├── INFORME_TECNICO.md
│   ├── MANUAL_OPERACIONES.md # Este archivo
│   └── ARQUITECTURA.md     # (Opcional)
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   └── test_main.py
└── .dockerignore
```

Dockerfile
docker-compose.yml
Jenkinsfile # Pipeline definition
README.md
requirements.txt

C. Checklist de Verificación

- ☐ Java JDK 17 instalado
- ☐ Jenkins instalado y ejecutándose
- ☐ Docker instalado y configurado
- ☐ Repositorio GitHub creado
- ☐ Jenkinsfile en repositorio
- ☐ Pipeline creado en Jenkins
- ☐ Webhook configurado en GitHub
- ☐ Credenciales de GitHub en Jenkins
- ☐ SMTP configurado para emails
- ☐ Primer build ejecutado exitosamente
- ☐ Aplicación desplegada y accesible
- ☐ Pruebas ejecutadas y pasadas
- ☐ Reportes generados correctamente
- ☐ Video demostrativo grabado y subido

Documento preparado por: Grupo D - Gestión de la Configuración del Software **Fe-**
cha: Mayo 2026 **Universidad:** Universidad de Guayaquil